

```
> ./pomotimer CLI
```

resource_path() + imports

```
1 import argparse, time, os, sys, subprocess
2 from tomato_art import TOMATO
3
4 def resource_path(relative_path):
5     base_path = getattr(sys, "_MEIPASS", os.path.abspath("."))
6     return os.path.join(base_path, relative_path)
```

Key Imports

- modules for command-line parsing (`argparse`)
- timing (`time`)
- OS navigation (`os, sys`)
- subprocess execution (`subprocess`)
- ASCII art from `tomato_art` file.

Resource Path Function

`resource_path` function resolves paths to bundled assets, working in both development (uses current directory) and frozen executables (uses PyInstaller's temp `_MEIPASS` dir via `sys._MEIPASS`).

How I compiled the binary →

```
pyinstaller --onefile \
--add-data
"tinker-ring.mp3:." \
--add-data
"tomato_art.py:." \ --name
pomotimer \ pomodoro.py
```

parse_time() + ANSI escape codes

```
8 RED = "\033[31m"
9 RESET = "\033[0m"
10
11 def print_tomato(color_on=True):
12     if color_on:
13         print(RED + TOMATO + RESET)
14     else:
15         print(TOMATO)
16
17 def parse_time(s):
18     s = s.strip().lower()
19     if not s:
20         raise ValueError("Empty time value")
21
22     if "h" in s:
23         h, m = s.split("h", 1)
24         hours = int(h) if h else 0
25         minutes = int(m) if m else 0
26         return float(hours * 60 + minutes)
27
28     if s.endswith("m"):
29         return float(s[:-1])
30
31     return float(s)
```

RED and RESET are ANSI escape sequences (`\033[31m` for red text, `\033[0m`

`print_tomato(color_on=True)` outputs the TOMATO ASCII art: in red if `color_on` is True (wrapping with RED + art + RESET),

Parse Time Function

`parse_time(s)` converts shorthand time strings (ex.: "0.5m", "2h30", "12") to minutes as a float

- Strips/lowercases input; errors on empty.
- Handles "XhY" -> X*60 + Y minutes.
- Handles "Xm" -> X minutes.
- Defaults to minutes if plain number.

play_sound() + time control functions

```
37 def play_sound():
38     return subprocess.Popen(
39         ["mpg123", "-q", resource_path("tinker-ring.mp3")],
40         stdout=subprocess.DEVNULL,
41         stderr=subprocess.DEVNULL,
42         stdin=subprocess.DEVNULL,
43     )
44
45 def start_timer(minutes, label, sound_on):
46     total_seconds = int(minutes * 60)
47     width = 20
48     for remaining in range(total_seconds, 0, -1):
49         elapsed = total_seconds - remaining
50         filled = int(width * elapsed / total_seconds)
51         bar = "-" * filled + " " * (width - filled)
52         mins, secs = divmod(remaining, 60)
53         print(f"\r[{bar}] {label}: {mins:02d}:{secs:02d}", end="", flush=True)
54         time.sleep(1)
55     bar = "-" * width
56     print(f"\r[{bar}] {label}: 00:00", end="", flush=True)
57     if sound_on:
58         play_sound()
59     input()
```

start_timer(minutes, label)

Converts `minutes * 60` to `total_seconds`. Uses `range(total_seconds, 0, -1)` countdown loop:

play_sound()

Uses `mpg123` to play the audio file, `-q` tells mpg123 to be quiet.

`DEVNULL`, disconnect the sound process so the process doesn't hang

UNIX-like help page (argparse)

```
57 def build_parser():
58     parser = argparse.ArgumentParser(
59         prog="pomotimer",
60         description="A simple CLI based pomodoro timer."
61     )
62     parser.add_argument("-w", "--work", default="25", help="work time, e.g. 0.5 or 25 or 1h45")
63     parser.add_argument("-b", "--break-time", default="5", help="short break time, e.g. 0.15 or 15m")
64     parser.add_argument("-l", "--long-break", default="15", help="long break time, e.g. 15")
65     parser.add_argument("-s", "--sessions", type=int, default=4, help="number of work sessions before the long break")
66     parser.add_argument("--no-color", action="store_true", help="disable colored output")
67     parser.add_argument("--sound", choices=["on", "off"], default="on", help="enable or disable sound")
68     parser.add_argument("-v", "--version", action="version", version="pomotimer 1.0.0")
69     return parser
```

build_parser()

This `build_parser()` function creates an `argparse.ArgumentParser` configured to auto-generate a Unix-style help page. Supposed to mimic a `unix help` page (almost) with a formatted usage synopsis, description, and options list with short and long flags. Goal was to make it intuitive for Unix users

Main function logic

```
71 def main():
72     parser = build_parser()
73     args = parser.parse_args()
74     color_on = not args.no_color
75     sound_on = args.sound == "on"
76
77     print_tomato(color_on)
78     print("Ctrl+C to exit")
79     print("pomotimer -h for help")
80
81     work_time = parse_time(args.work)
82     break_time = parse_time(args.break_time)
83     long_break_time = parse_time(args.long_break)
84     sessions = args.sessions
85
86     for i in range(sessions):
87         start_timer(work_time, "Work", sound_on)
88
89         if i < sessions - 1:
90             start_timer(break_time, "Short break", sound_on)
91
92     start_timer(long_break_time, "Long break", sound_on)
93     print("Pomodoro cycle finished.")
94
95 main()
```

main()

`main()` starts by parsing CLI args with `build_parser()`, sets `color_on = not args.no_color` and `sound_on = args.sound == "on"`, prints the tomato art plus the Ctrl+C/help notes, then converts the time arguments to minutes with `parse_time()`, runs a loop of work sessions each followed by a short break (except after the last one), then starts one final long break timer and finally prints "Pomodoro cycle finished.".